

Sprint Documentation

Sprint 15	Start Date: 07/04/2026	Final Date: 13/04/2026
Projetc Name:	Civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Wendell Alves Nascimento		
Gustavo Henrique Moreira Porto		

Sprint Backlog

Task#	Description	Start Date	Final date
#73	Adicionar busca de endereço automática por CEP	07/04/2026	13/04/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#73	Implementar a busca automática de endereço a partir do CEP , de forma genérica e reutilizável , permitindo seu uso em diferentes módulos do sistema (clientes, instituições, usuários, etc), melhorando a experiência do usuário e reduzindo erros de digitação.	Wendell Alves Nascimento, Gustavo Henrique Moreira Porto	Completo	10 horas	10 horas

Sprint Description

Tarefa: Sprint 15 - Adicionar busca de endereço automática por CEP

Objetivo

Implementar a **busca automática de endereço a partir do CEP**, de forma **genérica e reutilizável**, permitindo seu uso em diferentes módulos do sistema (clientes, instituições, usuários, etc), melhorando a experiência do usuário e reduzindo erros de digitação.

Critérios de Aceite

- Integrar com a API do ViaCEP para consulta de CEP.

Criar um serviço genérico e reutilizável para consulta de CEP.

Realizar busca automática ao inserir um CEP válido.

Preencher automaticamente os campos de endereço (logradouro, bairro, cidade, UF).

Permitir edição manual dos campos após preenchimento automático.

Tratar CEPs não encontrados.

Garantir que a aplicação não quebre em caso de falha na API.

Permitir reutilização da funcionalidade em diferentes entidades do sistema.

Atualizar documentação da funcionalidade.

Implementação deve ocorrer nas entidades **USUÁRIO, INSITUIÇÕES, FORNECEDOR e SECRETARIA**

Escopo da Implementação

A busca deve ser feita utilizando a API pública do ViaCEP:

<https://viacep.com.br/ws/{CEP}/json/>

Arquitetura esperada:

- Criar um **serviço (ex: CepService)** responsável pela consulta do CEP.
- Criar um **DTO padrão (ex: EnderecoDto)** para representar os dados retornados.
- O serviço deve ser independente de qualquer entidade específica.
- A lógica de consulta não deve ficar dentro de controllers ou entidades.
- O serviço deve ser injetado via **Dependency Injection**.

Exemplo de resposta da API:

```
{  
  "cep": "15700-200",  
  "logradouro": "Avenida João Amadeu",  
  "bairro": "Jardim Samambaia",  
  "cidade": "Jales",  
  "uf": "SP"  
}
```

Fluxo esperado:

1. Usuário insere o CEP.
2. Sistema valida o formato do CEP (apenas números, 8 dígitos).
3. Ao validar, realiza requisição para a API através do serviço.
4. Caso sucesso:
 - Preenche automaticamente os campos.
5. Caso erro:
 - Exibe mensagem informando CEP inválido ou não encontrado.

Observações

- Implementar tratamento para o campo "erro": true retornado pela API.
- Evitar múltiplas requisições desnecessárias (debounce opcional).
- A funcionalidade deve ser desacoplada e reutilizável em qualquer formulário.
- Em caso de indisponibilidade da API, permitir preenchimento manual normalmente.
- Evitar duplicação de código entre módulos.
- Analise todo o projeto antes do envio

Sprint Results

Foi implementada a funcionalidade de consulta de endereço por CEP de forma genérica e reutilizável no backend, sem alterações em entidades ou no banco de dados. Foram criados o DTO `EnderecoViaCepDTO.cs`, a interface `ICepService.cs`, a exceção de infraestrutura `CepServiceException.cs` e o serviço `CepService.cs`, responsável por sanitizar o CEP, validar o formato com 8 dígitos, consultar a API ViaCEP e diferenciar cenários de CEP inexistente e falha externa.

Também foi adicionado o endpoint protegido `GET /api/cep/{cep}` em `CepController.cs`, seguindo o envelope padrão `Response`, com retorno 200 para sucesso, 400 para CEP inválido, 404 para CEP não encontrado e 500 para erro na integração. No runtime, `Program.cs` foi atualizado para registrar `AddHttpClient<ICepService, CepService>()`, e `appsettings.json` passou a incluir a configuração `ViaCep` com `BaseUrl` e `timeout`.



Na cobertura de qualidade, foi criado `CepEndpointsTests.cs` com testes de integração para os cenários de sucesso, CEP com máscara, CEP inválido, CEP não encontrado, falha externa e acesso sem token, utilizando `HttpClient` mockado, sem dependência da API real. Também foi ajustado `FornecedorValidationEndpointsTests.cs` para manter a suíte compatível com autenticação nas rotas protegidas. Por fim, `README.md` foi atualizado para documentar o novo endpoint e seu uso no preenchimento automático de endereço.

Assinaturas

Documento assinado digitalmente
 GUSTAVO HENRIQUE MOREIRA PORTO
 Data: 10/04/2026 13:15:47-0300
 Verifique em <https://validar.iti.gov.br>

Documento assinado digitalmente
 WENDELL ALVES NASCIMENTO
 Data: 10/04/2026 13:46:44-0300
 Verifique em <https://validar.iti.gov.br>